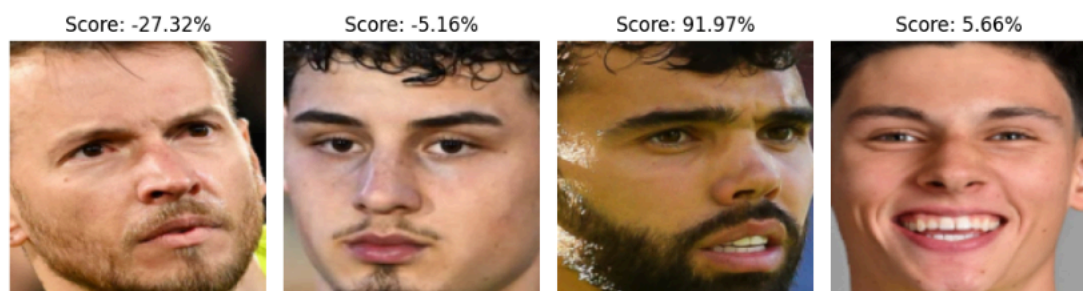


3a Design Report

27.06.2025 - **Week 1**

To build a system capable of identifying players from multiple view angles, MTCNN (Multi-Task Cascaded Convolutional Networks) was used for face detection, and InceptionResnetV1 for generating facial embeddings. The system processes images from URL links, detecting faces and converting them into vector representations (embeddings). A target image is compared against several candidate images by measuring cosine similarity between their embeddings. This method successfully identifies the most similar face with high accuracy, providing a strong foundation for further application development.

```
most_similar_image, similarity_score = find_most_similar(image_url_target, candidate_image_url)
print(f"The most similar image is: {most_similar_image}")
print(f"Similarity score: {similarity_score * 100:.2f}%")
```



The most similar image is: https://e0.365dm.com/23/02/2048x1152/skysports-david-raya-brentford_6052925.jpg?20230210145620
Similarity score: 91.97%

MTCNN and InceptionResnetV1

04.07.2025 - **Week 2**

Testing MTCNN for face detection in video was the next step in exploring the capabilities of the system. Presenting this work to the group sparked discussions around creating player templates to improve identification accuracy, distinguish between teams, and preserve player individuality.



Face detection - Processed video

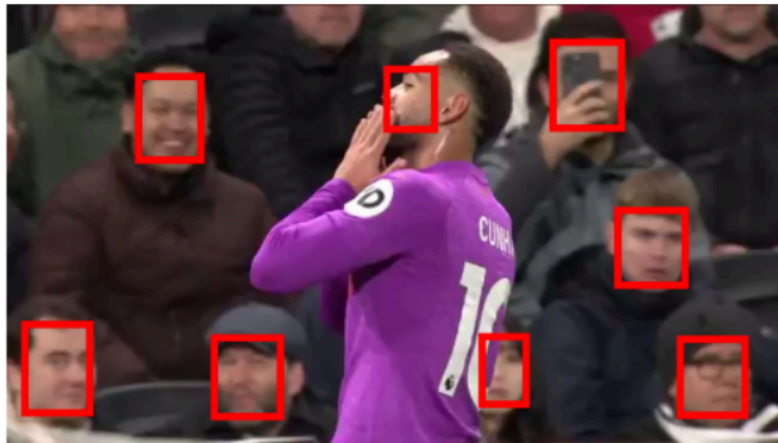
11.07.2025 - **Week 3**

As the software developed further, the goal became to build an optimal system for face detection and player identification that not only performed accurately, but could also be adapted for multiple use cases. Building on earlier work, player faces were detected and facial embeddings were extracted, capturing key features unique to each individual. These embeddings were used to create player templates, allowing the system to compare a target face against known profiles to identify the most similar match.

Ideally, this system would store facial data for all team members, officials, and other relevant individuals, enabling real-time identification during live footage by continuously comparing detected faces against the stored templates.

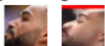
Following a meeting with David Atkins, it became clear that the software's performance—particularly in terms of accuracy, precision, and speed—is critical. Therefore, the next stage of development will focus on extracting performance metrics and optimising the comparison pipeline for faster, more efficient real-time detection and identification.

Player 0 - Template: [0.0148074 -0.00828731 -0.01547996 0.02705143 -0.01679516]...

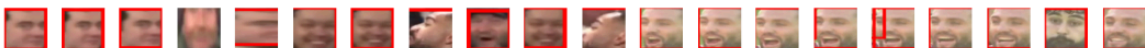


Processed frame extracting embeddings

Target Image similar



Score: 9.0176% 5.4092% 6.0079% 2.055% 5.1024% 7.6.012% 5.7.05% 8.0068% 5.7007% 4.2021% 1.8008% 2.6097% 8.0071% 3.5090% 2.7097% 5.7074% 3.8099% 6.5018% 3.2062% 42.70



The most similar image is: C:/Users/user/Documents/Projects/Internship/faceNet/processed_frames/Frame_8.png
Similarity score: 43.68%

Similarity score - Target image compared against player templates

18.07.2025 - **Week 4**

This week was focused on restructuring the face detection and matching pipeline using MTCNN, with promising results as detection now runs in under one second per frame. However, detection time increases with the number of faces per frame, and bounding box sizes from MTCNN remain inconsistent—often cropping too tightly around facial features—making it unreliable to use box size alone for filtering. A "Find Face > Run Match" approach was tested but faced issues with processing time exceeding 20 minutes per video and frequent false negatives ("No matches detected") despite previous successful matches.

Currently using 360p resolution works for detection, but certain frames cause failures due to distortion or tight crops, highlighting the need for a data cleaning process. A pre-matching cleaning step would help remove poor-quality frames and normalise inputs, improving both

accuracy and efficiency. Testing on higher resolution video is also recommended. Building on the structure created in Week 2 still seems viable—it works, but needs refining for speed and reliability. Although the current method doesn't involve training a model, it's producing usable results, so the focus should remain on improving data handling rather than retraining. Real-time detection continues to be a concern, but a better data cleaning process could significantly reduce the issues currently slowing down the system.

25.07.25 - **Week 5**

This week, I explored image matting, focusing on finding software that offers a fully automated process for extracting foregrounds from backgrounds using image input. I reviewed several repositories and eventually found a research paper that led to promising software. Initial tests produced undesirable results, but after isolating a different model, output quality improved—though not yet to the level of accuracy we need. I'm currently unsure if it's worth continuing down this path, as results are inconsistent. We're now halfway through the project (Week 6), and I'm considering whether expanding on our original concept may ultimately prove more effective. That concept involves creating a group of known faces (team players), assigning each player three facial angles, using those to form a player template, detecting faces in video, and flagging any with high similarity to a stored template. Concerns with this approach remain: cycle speed, handling unwanted faces, avoiding mismatches, determining whether the software truly recognises individuals, and evaluating detection performance with supporting data.

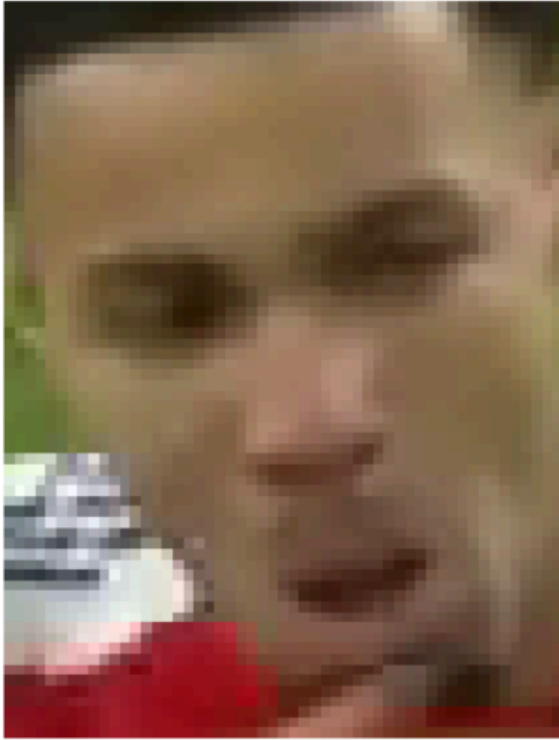


01.08.25 - **Week 6**

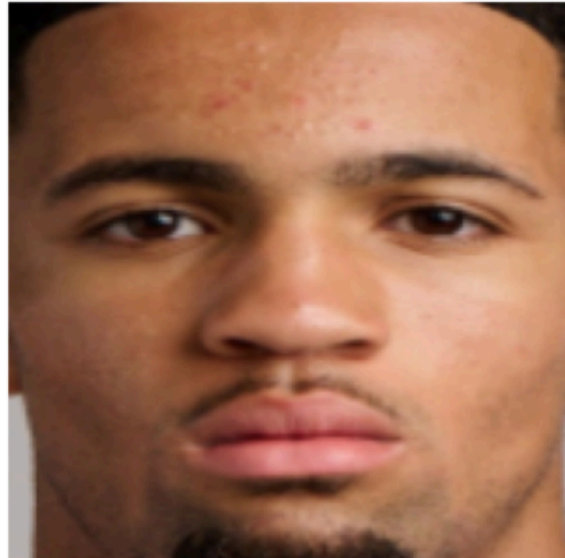
The face detection process has been refined, with player faces now being successfully identified and compared for similarity using facial embeddings and cosine similarity. While the system performs effectively in most scenarios, a key limitation remains: difficulty in

detecting or accurately matching side-profile faces. It is still unclear whether the current embedding model sufficiently accounts for these angled views. Further research is underway to address this issue, as it is the main factor currently limiting improvements in face detection and identification accuracy.

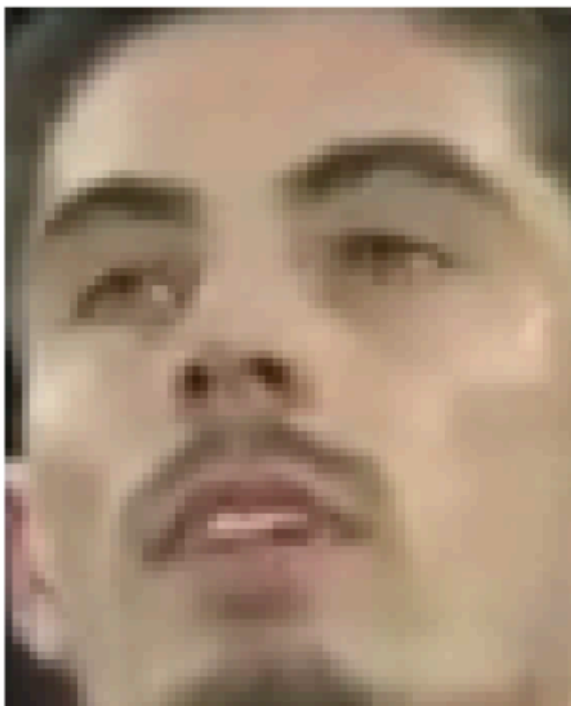
Detected Face



Matched: player26



Detected Face



Matched: player32



08.08.25 - Week 7

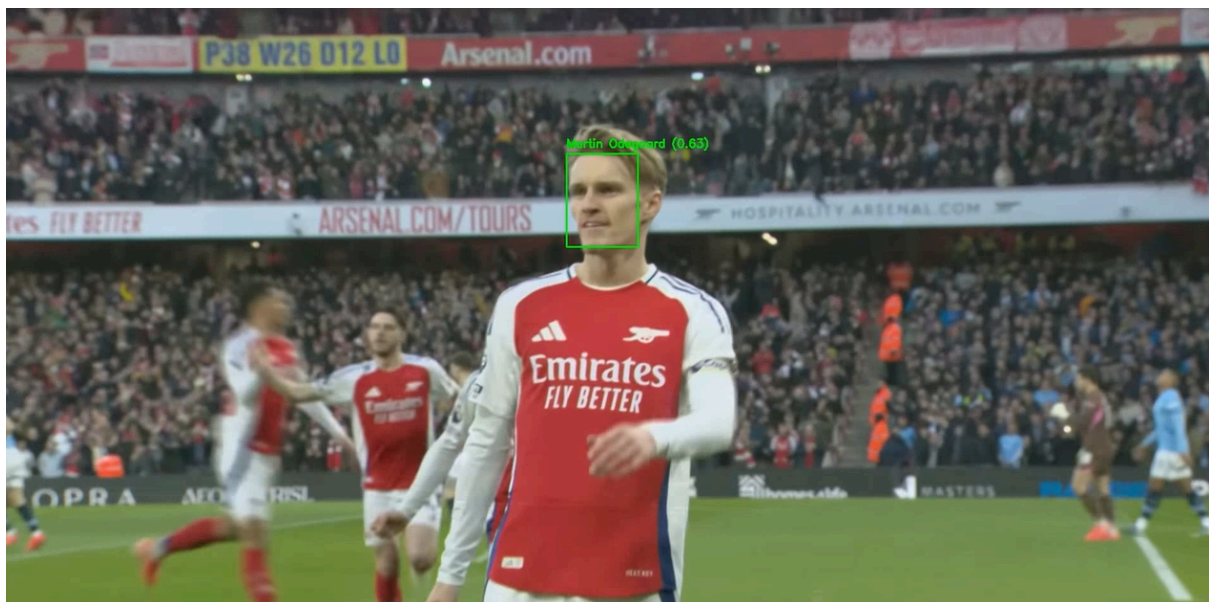
I worked on refining the face detection and identification software, implementing an anchor-based comparison method for embedded images. The system now stores the first detection only if its similarity score is close to a defined threshold, using it as a reference template. Every subsequent detection of the same person is matched against both this initial template and their embeddings, which should improve accuracy. I also collected 10 images for each player and reprocessed them through the embedding and matching pipeline to strengthen recognition performance.

Pros:

- Face detection/identification prototype complete.
- Capable of detecting multiple players in a frame, including side-profile views.
- Successful detection possible with adequate player angle coverage.

Cons:

- Need to test the multiple-angle detection theory further.
- Does not capture fast appearance flashes—unclear if this is an issue.
- Low similarity threshold (0.6) indicates uncertainty, possibly due to embedding quality (bad input = bad output).
- Processing speed is slow; could be improved with a search algorithm instead of the current brute-force method.



Face system prototype



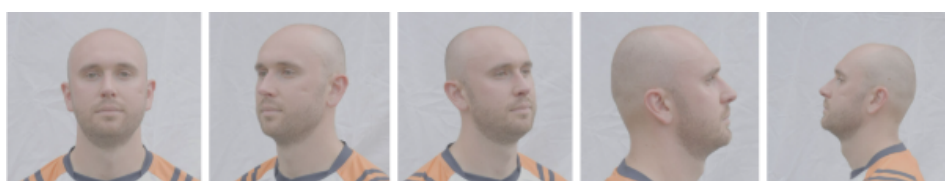
Face system prototype - Side profile detected



Face system prototype - Angled face detected

15.08.25 - **Week 8**

After completing the face system prototype, I began researching search algorithms to improve the speed of the recognition process. During this research, I identified FAISS as a promising option and have started implementing it into the project, though results are still pending. This week was also spent building a dataset for the system by collecting images at a hockey game in Bury St Edmunds, where multiple angled photos were taken of each player. These images will be processed through the face system to test and improve the matching accuracy.



22.08.25 - Week 9

This week was spent processing the images collected from the hockey games. The images were run through the system, where they were cropped and resized in preparation for face matching. In addition, an all-in-one script was developed, enabling the software to function as a simple push-button tool that takes input and produces ready-to-use output for handover.

```
# --- STEP ONE : ALIGN FACE ---
def align_faces(input_root="face_detection", output_root="aligned_faces"):
    input_root = Path(input_root)
    output_root = Path(output_root)

    for team_dir in input_root.iterdir():
        if team_dir.is_dir():
```

```
# --- STEP TWO : CREATE EMBEDDING ---
def create_embeddings(input_root="aligned_faces"):
    input_root = Path(input_root)

    for team_dir in input_root.iterdir():
        if team_dir.is_dir():
```

```
# --- STEP THREE : CREATE PLAYER TEMPLATE ---
def create_templates(embedding_root="aligned_faces", template_output_root="player_templates"):
    embedding_root = Path(embedding_root)
    template_output_root = Path(template_output_root)
    template_output_root.mkdir(exist_ok=True)

    for team_dir in embedding_root.iterdir():
        if team_dir.is_dir():
```

```
# --- STEP FOUR : FIND MATCHES ---
anchors = {}

def match_face_to_players_with_anchor(face_embedding, templates, threshold=0.6, anchor_threshold=0.8):
    best_score = -1
    best_player = "Unknown"

    for player, views in templates.items():
```

29.09.25 - Week 10

This week was focused on exploring search algorithms to improve the processing speed of the face recognition system. I spent time reviewing a number of GitHub repositories to establish a foundation for development, comparing different approaches and their potential trade-offs. During this research, I came across FAISS, a library developed by Facebook that is widely used for fast similarity search and clustering of dense vectors. Given its reputation for efficiency and scalability, FAISS seemed like a strong candidate for integration. I have begun implementing it into the project with the expectation that it will significantly reduce search times and make the system more robust when handling larger datasets. While results are still to be validated, this represents a promising step forward in optimising performance.

05.09.25 - Week 11

After implementing FAISS, I observed that while the processing speed did significantly improve, the integration into my existing codebase proved problematic. The system began generating false matches similar to those encountered in earlier stages of the project, which undermined the reliability of the results. Additional complications also arose, including unexpected terminal crashes that disrupted the workflow. Given the limited time remaining in the project, it became clear that fully integrating FAISS would require more extensive research, testing, and adaptation than the current timeline allows. As a result, the decision was made to step back from using FAISS for now, and instead focus on stabilizing and refining the existing system to ensure a reliable handover.

12.09.25 - **Week 12**

This week was focused on final polishing of the code and preparing the FaceSystem project for handover. The completed project was uploaded to the 3A design system, and all associated work was formally passed on to David and Simon. In addition, the full week-by-week project report was submitted to provide a clear record of progress and development throughout the project..